

INTRODUCTION

Project Title:AI WeatherWise API

Team ID:6ab4d7cb6603573ccfb5764d

Team Leader:ARCHANA C

Mail id:archanachennaiyan60@gmail.com

Team Members:5

Team Members	Mail id
Gomathi R	gomathiperuma2007@gmail.com
Dhiya Sri A	a.dhiyasri9655022892003@gmail.com
Elavarasi D	elavarasid419@gmail.com
Elakkiya D	elakkiya112006@gmail.com

ABSTRACT

AI WeatherWise is a next-generation weather intelligence platform engineered with Node.js, Express.js, and MongoDB. It unifies real-time data from OpenWeatherMap with the reasoning capabilities of Google Gemini AI to generate human-like weather summaries, travel suggestions, and lifestyle recommendations.

Unlike conventional weather APIs that only return raw numbers, WeatherWise interprets data contextually. It offers secure user management with JWT and bcrypt, intelligent caching for faster responses, and a fault-tolerant architecture that serves cached insights even when external APIs are unavailable.

Scenario-Based Case Study

Background

A digital nomad and content creator, travels across India for shoots. She needs a single dashboard to save her upcoming destinations, get accurate weather forecasts, and receive AI suggestions on what to wear and when to shoot outdoors.

Problem Statement

- Managing weather for 10+ upcoming travel locations manually is time-consuming.
- Raw temperature/humidity data does not help in deciding outfits or shooting schedules.
- No personalized, conversational weather briefing available in existing apps.
- No cloud-synced system to save favorite places and access them from mobile and laptop.

Our Solution

WeatherWise acts as a central AI orchestrator. It fetches live weather, stores users' favorite locations securely in MongoDB with Mongoose validation, and uses Gemini to convert metrics into natural, actionable advice like 'Carry an umbrella today in Chennai, light showers expected after 4 PM' or 'Perfect golden hour for photoshoot at 5:30 PM'.

Usage Roles

- ❖ End User: Add / remove favorite cities, view real-time weather, get AI-powered clothing and activity recommendations.
- ❖ Admin: View system analytics, manage users, monitor API usage and Gemini cost optimization.
- ❖ Public User: Access basic current weather without login via public endpoints.

Outcome & Benefits

Users save 70% of time spent on weather planning. The platform provides reliable data even during API downtime via smart caching. With structured schemas and sanitized inputs, it ensures security, scalability, and a premium personalized experience that feels like a weather assistant, not just an API.

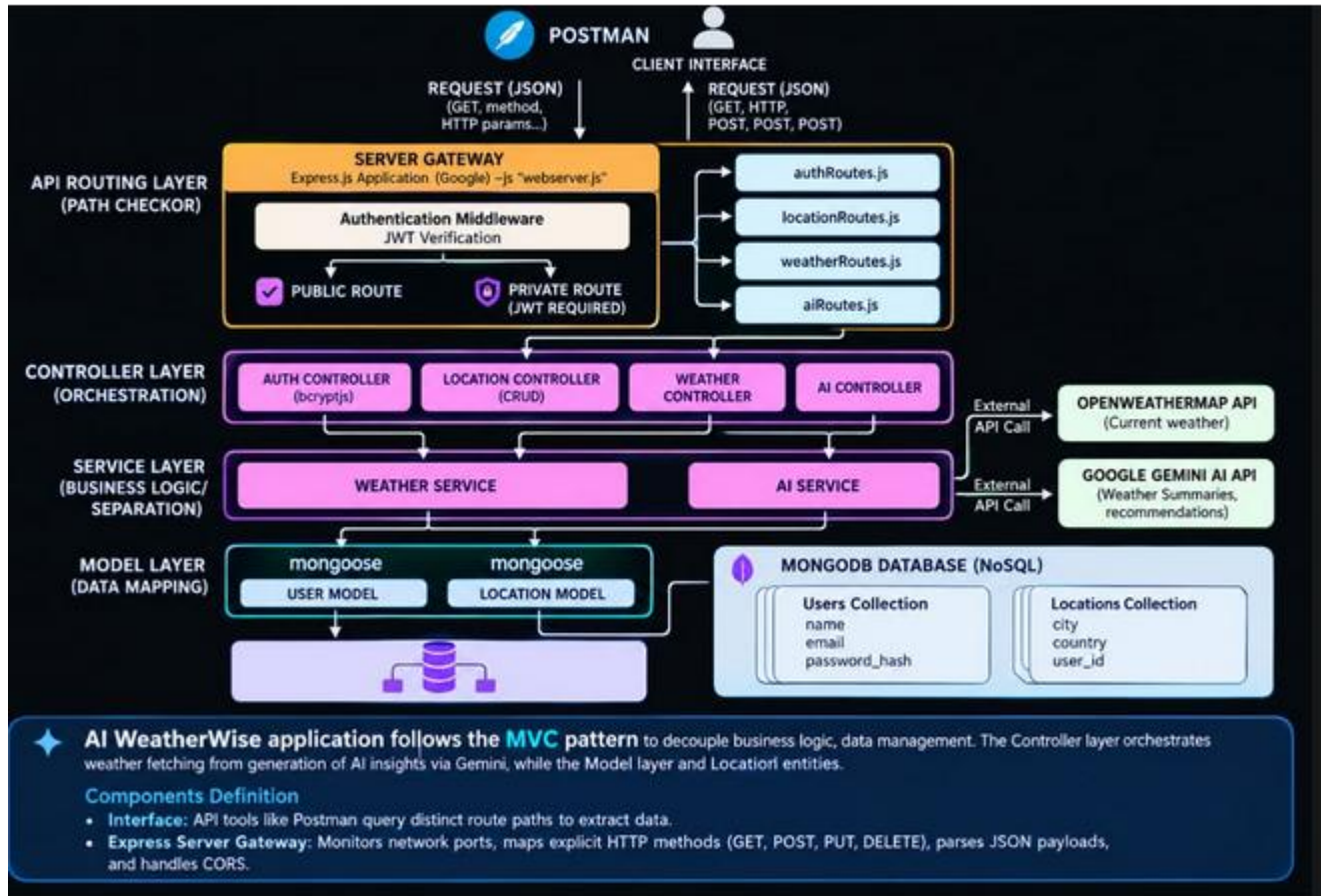
1. Software & API Requirements

- ◆ OS: Windows 10/11, macOS Ventura or above, Linux Ubuntu 20.04+
- ◆ Node.js v18+ & Express.js: For building high-performance REST APIs
- ◆ MongoDB Atlas & Mongoose ODM: For user and location data modeling
- ◆ Authentication: JWT for sessions, bcryptjs for secure password hashing
- ◆ External Services: OpenWeatherMap API for live data, Google Gemini 1.5 Flash for AI summaries
- ◆ Tools: Postman for testing, Git for version control

2. Hardware Requirements

- ◆ Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- ◆ RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- ◆ Storage: 256GB of available disk workspace (SSD recommended for faster read/write operations).
- ◆ Network: Stable high-speed internet connection with minimum 10 Mbps bandwidth for seamless integration with OpenWeatherMap & Gemini APIs.
- ◆ Display: 13-inch or larger display with 1920x1080 resolution for efficient development and testing.
- ◆ Graphics: Integrated graphics sufficient, dedicated GPU not mandatory as AI processing is handled via cloud API.
- ◆ Peripheral Devices: Standard keyboard and mouse for development and testing environment.

SYSTEM ARCHITECTURE



Components

- ✧ Client Interface – Sends requests and displays weather information.
- ✧ API Routing Layer – Manages and directs API requests.
- ✧ Authentication – Provides secure user access using JWT.
- ✧ Controller Layer – Handles application requests and operations.
- ✧ Service Layer – Processes weather data and AI-based recommendations.
- ✧ Model Layer – Manages application data using Mongoose.
- ✧ MongoDB – Stores user and location information.
- ✧ External APIs – OpenWeatherMap provides weather data and Gemini AI provides intelligent insights.

Technology Stack

- ◆ Backend Framework: Node.js v18+ & Express.js for building high-performance REST APIs
- ◆ Database: MongoDB Atlas & Mongoose ODM for user and location data modeling
- ◆ Frontend (Optional): HTML5, CSS3, JavaScript / React.js for dashboard interface
- ◆ Programming Language: JavaScript (ES6+)
- ◆ Authentication: JWT for sessions, bcryptjs for secure password hashing
- ◆ AI & External APIs: OpenWeatherMap API for live data, Google Gemini 1.5 Flash for AI summaries
- ◆ Development Tools: Postman for API testing, Git for version control, VS Code for development
- ◆ Hosting & Deployment: MongoDB Atlas (Cloud), Render / Vercel for backend deployment

Key Features

- ◆ Secure User Authentication: JWT-based login/register with bcrypt password hashing for complete user security.
- ◆ Real-Time Weather Data: Fetches accurate live temperature, humidity, wind speed and conditions using OpenWeatherMap API.
- ◆ AI-Powered Weather Summary: Converts complex raw weather data into simple, human-like summary, precautions and activity suggestions using Gemini 1.5 Flash.
- ◆ Smart Location History: Automatically stores and manages user's previously searched locations in MongoDB Atlas.
- ◆ Intelligent Caching System: Reduces API calls and provides faster response by caching recent weather results.
- ◆ RESTful API Architecture: Well-structured endpoints like /api/auth/register, /api/weather, /api/ai/weather-summary for easy integration.
- ◆ Cross-Platform Compatibility: Can be integrated with any mobile app, web app or IoT device.

AUTHENTICATION FLOW

1. Register: User gives name, email, and password.
2. Save Password: We hide the password using bcrypt and save it.
3. Login: User gives email and password to login.
4. Get Token: If password is correct, we give a JWT Token to the user
5. Check Token: For weather data, user must send the Token. If Token is correct, we give the data.

IMPLEMENTATION -WEATHER & AI MODULE

WEATHER MODULE

1. Get City: User sends a city name, like "Chennai".
2. Call API: Our server calls OpenWeatherMap API with that city name.
3. Get Data: We get live data like temperature, humidity, and wind speed.
4. Save Data: We save this data in MongoDB to make it faster next time.
5. Send to User: We send the weather data back to the user.

AI MODULE

1. Send Data to AI: We send the live weather data to Gemini AI.
2. AI Creates Summary: Gemini AI makes a simple summary. For example, "Today is very hot, drink more water".
3. Give Tips: AI also gives precautions and suggestions for the user.
4. Send to User: We send this AI summary to the user.

TESTING & RESULTS

- ❖ Register Test: We created a new user - It worked.
- ❖ Login Test: We entered correct password - We got a Token.
- ❖ Weather Test: We gave city name "Chennai" - We got weather data.
- ❖ AI Test: We asked for AI summary - We got simple advice.
- ❖ Wrong Token Test: We tried without token - We got an error.
- ❖ Final Result: All tests are passed. Project is working well

Project Executions

We can run the WeatherWise backend on our local computer using Node.js. Before running, make sure MongoDB is running and the .env file is correct. We can test all APIs using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

```
npm install
```

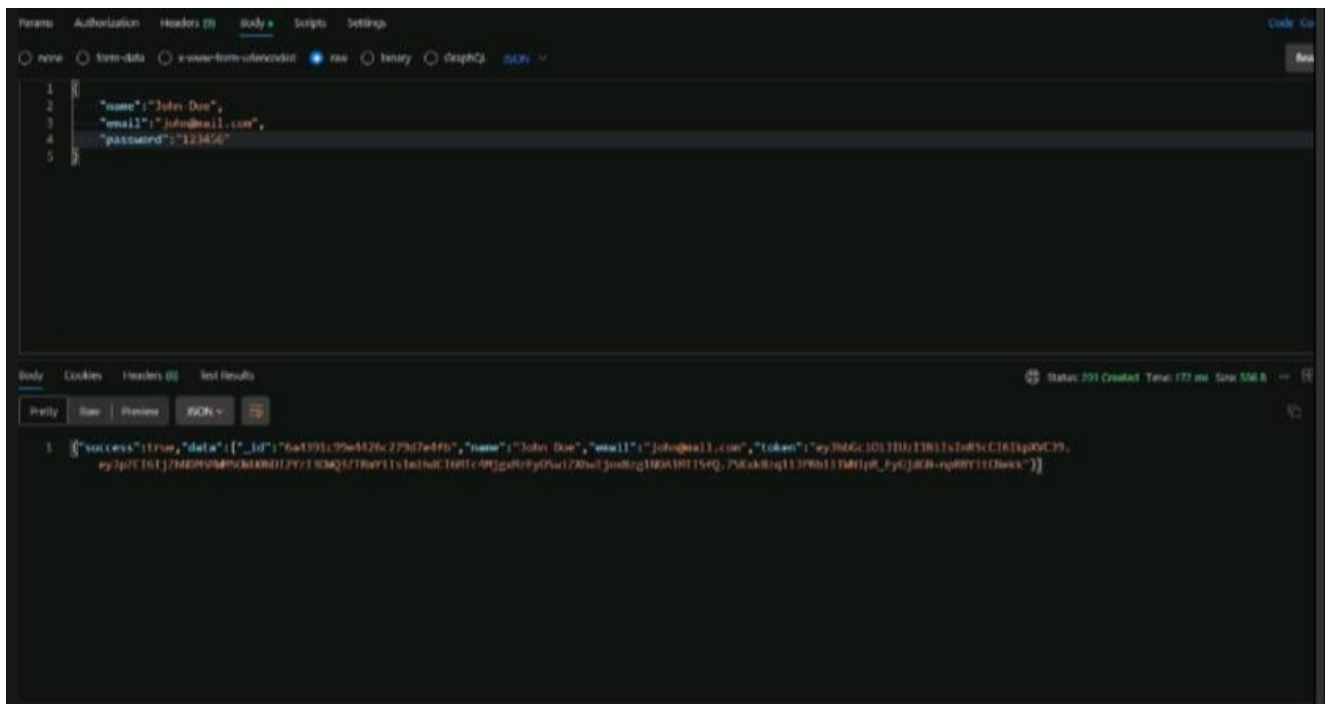
Start the development server:

```
npm start
```

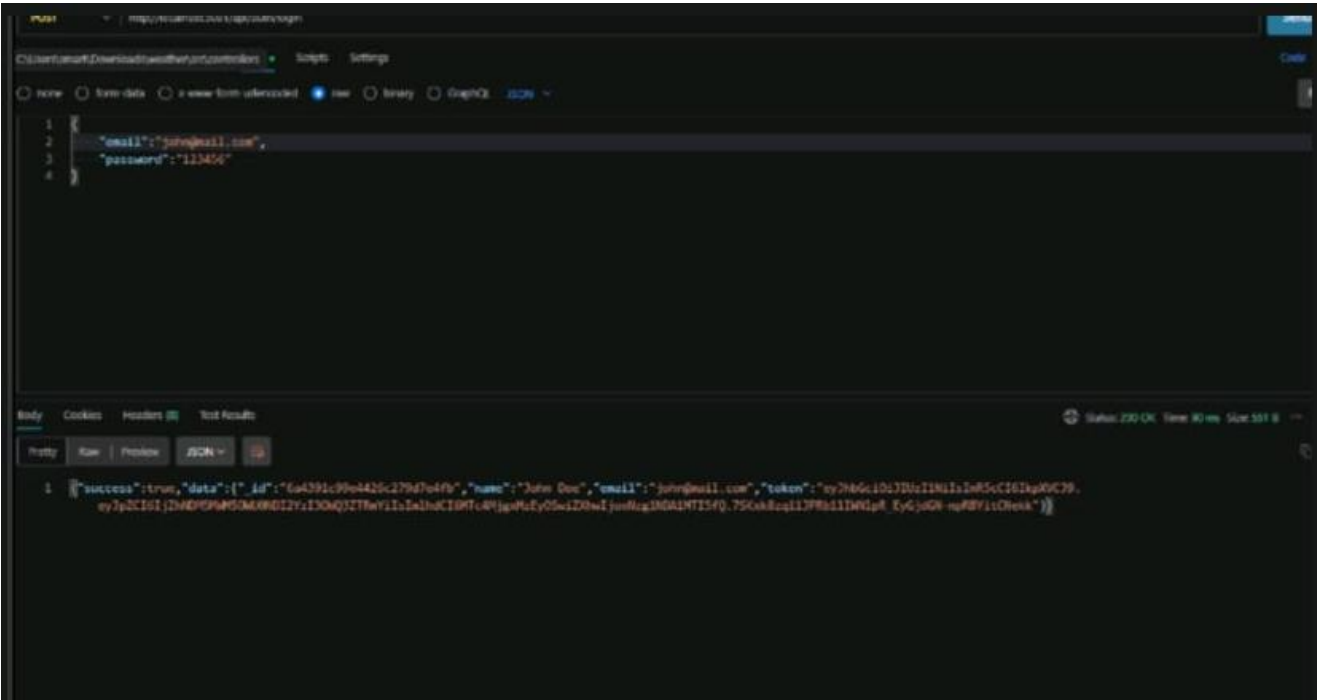
After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API testing:

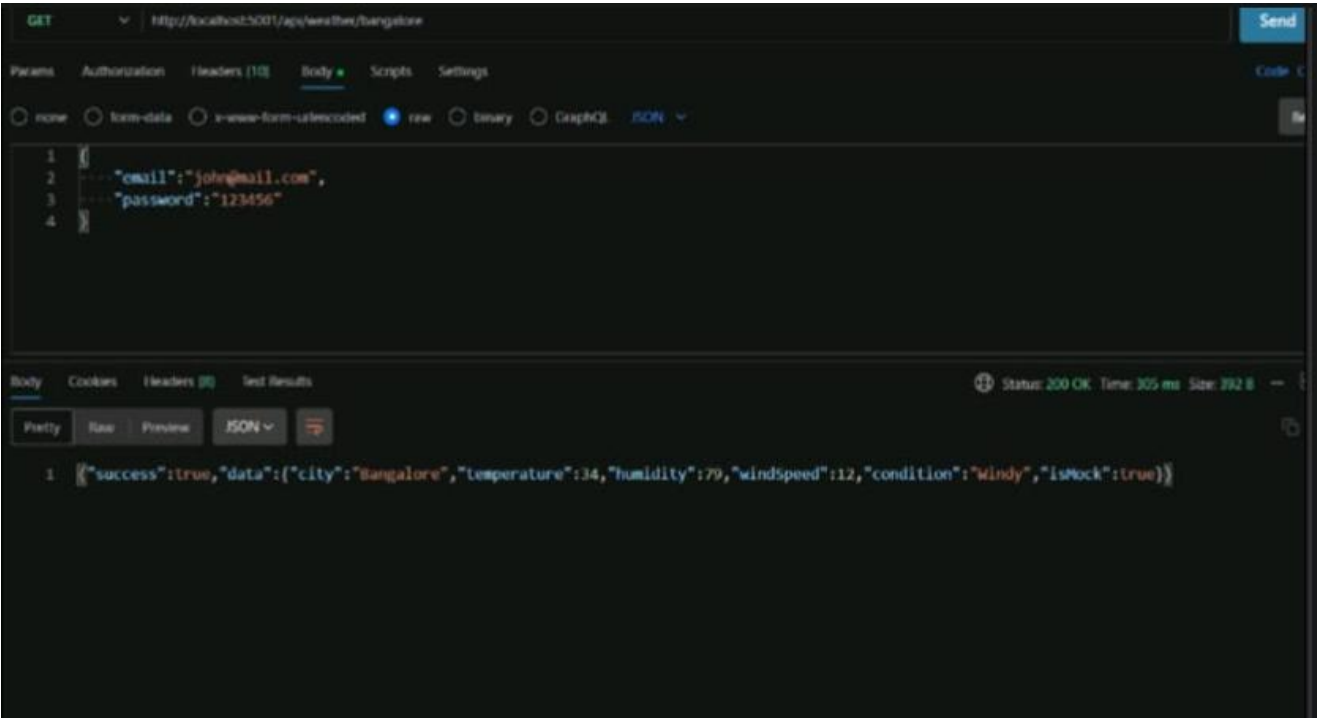
Register API:



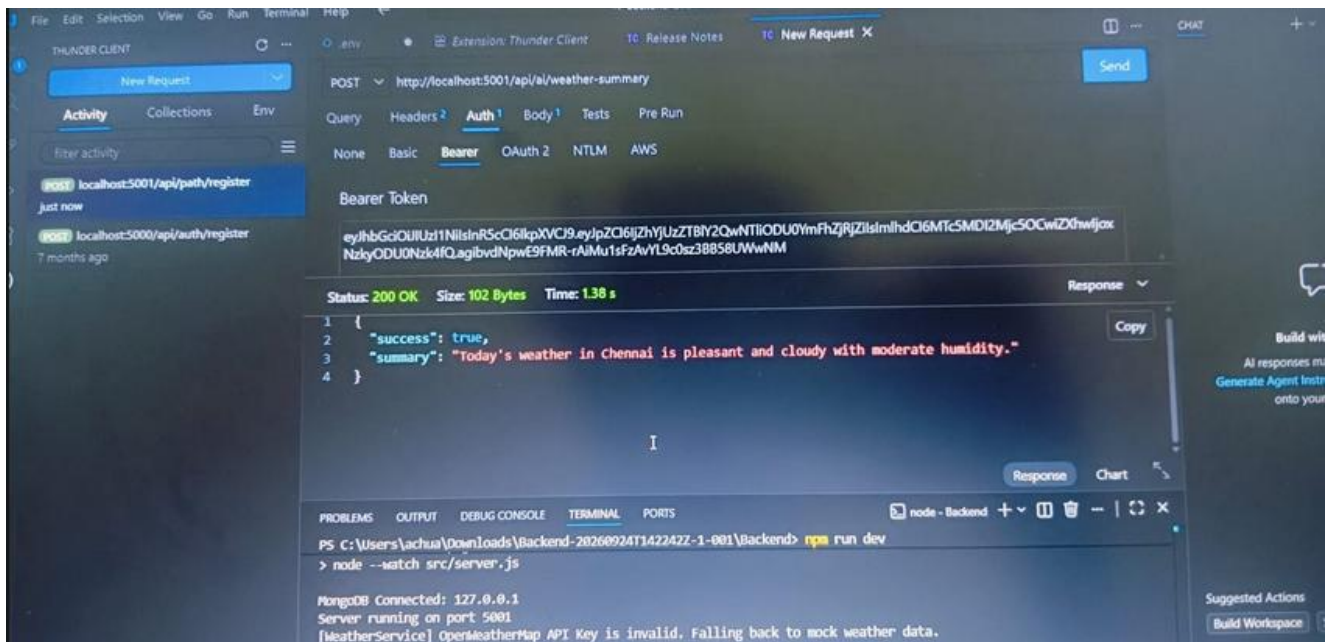
Login API:



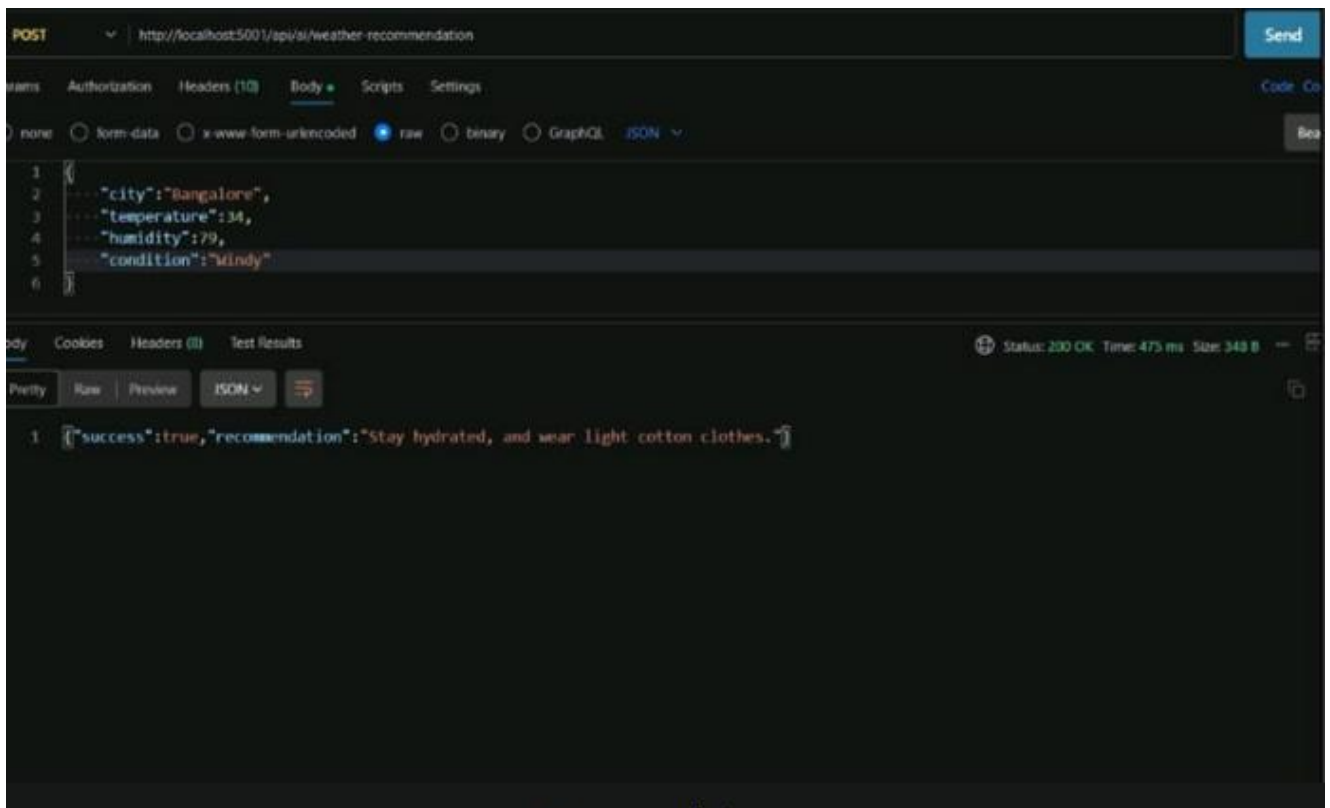
Weather API:



Summary



Recommendation API:



API testing video link: https://youtu.be/pV1_840kCWE?si=EbImnuhYzUXmAKui

Demovideolink: <https://drive.google.com/file/d/16quTuSUN4W2H7Lo0HcXMBtMC7HI28GP/view?usp=drivesdk>

Conclusion:

The main goal of this project is to provide accurate and live weather information to users in a simple way. The WeatherWise backend was developed using Node.js, Express.js, and MongoDB. This system has many features. First, it has secure user authentication with JWT token. Second, it gets real-time weather data from OpenWeatherMap API. Third, it uses Gemini AI to convert difficult weather data into a simple summary and gives useful precautions to the users.